

MemCache-Ejercicio-1.pdf



Pausevi



Estructura de Computadores



2º Grado en Ingeniería Informática



**Facultad de Informática
Universidad Complutense de Madrid**

Ejercicio 1. En un computador con caches separadas de datos e instrucciones, de 32KB cada una y bloques de 256 bytes, se quiere ejecutar el siguiente código C, que realiza la operación matricial $C = A+B$:

```
#define N 128
int A[N][N];
int B[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = A[i][j] + B[i][j];
```

Asumiendo que la matriz **A se ubica en la dirección 0x0C000000**, que B y C se almacenan consecutivamente a continuación de A, y que en la traducción a ensamblador del código anterior la variable *i* se ubica en un registro y que las direcciones de comienzo de los *arrays* están también en registro, se pide:

a) Calcular el número de fallos de cache en lectura (load) y en escritura (store) si la cache es de emplazamiento directo sin asignación en escritura.

b) Repetir el cálculo para una cache asociativa por conjuntos, de 2 vías, con política de reemplazamiento LRU, en los siguientes casos:

sin asignación en escritura.

con asignación en escritura.

En cada caso indicar si el rendimiento mejoraría o empeoraría y en qué proporción.

c) Un programador sugiere la siguiente modificación de código para una cache asociativa por conjuntos de 2 vías con asignación en escritura:

```
#define N 128
struct {
    int A;
    int B;
} AB[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = AB[i][j].A + AB[i][j].B;
```

Asumiendo que el array AB se ubica en la dirección 0x0C000000 y que el array C se ubica a continuación, calcule el número de fallos en lectura y escritura que se producirían, y compárelo con el resultado del apartado b para la misma configuración de cache.

d) ¿Puede obtenerse un resultado similar al del apartado c) con el código original y sin aumentar el tamaño de la cache? Razone la respuesta.

a) emplazamiento directo

No nos dicen el tamaño de la memoria principal, pero nos indican que la dirección donde se coloca la matriz A en memoria principal es 0x0C00 0000 por lo que podemos intuir que tenemos una memoria de 4 Gbytes

Memoria cache de 32 kbytes = 2^{15} bytes

Como las líneas (o bloques) son de 256 bytes --> 8 bits para determinar byte de la línea.

Total de líneas es $2^{15}/2^8 = 128$ líneas --> 7 bits para direccionar la línea:

Etiqueta	Línea de Cache	Selección de Byte
17	7	8

Nos dicen que tenemos cache de datos e instrucciones y nos preguntan por los fallos en la cache de datos.

Las matrices A, B y C son de igual tamaño.

Contienen 128×128 enteros (de 4 bytes) $\Rightarrow 2^7 \times 2^7 \times 2^2 = 2^{16}$ bytes

Esto nos indica que cada una de las matrices llena por completo la cache dos veces.

Nos indican que la primera matriz, A, se coloca a partir de la dirección de MP:

0x0C00 0000

Como ocupa 64 kbytes y la matriz B comienza a continuación sabemos que B comenzará a partir de la dirección:

0x0C01 0000

Y la matriz C comenzará en:

0x0C02 0000

Etiqueta	Línea de Cache	Selección de Byte
17	7	8

```
for (i=0; i < N; i++)  
    for (j=0; j < N; j++)  
        C[i][j] = A[i][j] + B[i][j];
```

Las referencias A[0][0-63] corresponden a la primera línea de la cache

Las referencias B[0][0-63] corresponden también a la primera línea de la cache

Las referencias C[0][0-63] corresponden también a la primera línea de la cache

```
for (i=0; i < N; i++)  
    for (j=0; j < N; j++)  
        C[i][j] = A[i][j] + B[i][j];
```

Las referencias A[0][0-63] corresponden a la primera línea de la cache

Las referencias B[0][0-63] corresponden también a la primera línea de la cache

Las referencias C[0][0-63] corresponden también a la primera línea de la cache

Al referenciar A[0][0] se produce un FALLO y se trae el bloque a la línea 0.

A continuación se referencia B[0][0] que también produce FALLO y se trae al bloque 0 reemplazando a lo que allí había.

Al referenciar C[0][0] como es un FALLO para escritura no se trae el bloque a cache sino que se escribe directamente en MP.

Al referenciar A[0][1] se vuelve a traer el bloque a cache reemplazando a B y así sucesivamente.

Por tanto todas las referencias a A y B son fallos de bloque: $128 \times 128 \times 2 = 32768$ fallos

En cuanto a las escrituras todas producen fallo, porque ninguna están en cache, pero no se trae ningún bloque, simplemente son 16.384 escrituras en Memoria Principal.

b) Repetir el cálculo para una cache asociativa por conjuntos, de 2 vías, con política de reemplazamiento LRU, en los siguientes casos:

sin asignación en escritura.

con asignación en escritura.

En cada caso indicar si el rendimiento mejoraría o empeoraría y en qué proporción.

b.1. Sin asignación en escritura

Al ser asociativa por conjuntos con dos vías los 128 bloques se dividen en 64 conjuntos con dos bloques cada uno.

Etiqueta	Conjunto de Cache	Selección de Byte
18	6	8

Las referencias A[0][0-63] corresponden al primer conjunto de la cache

Las referencias B[0][0-63] corresponden también al primer conjunto de la cache

Las referencias C[0][0-63] corresponden también al primer conjunto de la cache

Pero ahora el bloque que contiene los elementos A[0] [0-63] se coloca en el conjunto 0, bloque 0;
mientras el bloque que contiene los elementos B[0] [0-63] se coloca en el conjunto 0, bloque 1

Las referencias a C siguen actualizándose sólo en MP.

Cada vez que traemos un bloque se produce un fallo de bloque y después 63 aciertos, por tanto el número de fallos se divide por 64:

Fallos de lectura = $32768 / 64 = 512$

16.384 escrituras en Memoria Principal.

b.2. Con asignación en escritura

En un fallo de escritura se trae el bloque donde se ha producido el fallo a la cache

```
for (i=0; i < N; i++)  
    for (j=0; j < N; j++)  
        C[i][j] = A[i][j] + B[i][j];
```

Por tanto como sólo hay dos vías cada iteración se leen o escriben elementos de las tres matrices con el mismo índice que corresponderán al mismo conjunto y al haber sólo dos líneas por conjunto producirán fallos continuados:

Fallos = $128 * 128 * 3 = 49.152$

Mejora del rendimiento

2ª. Sin asignación en escritura

$$\text{Speed_up} = T_{\text{empl_directo}} / T_{\text{empl_asociativo}}$$

$$T_{\text{empl_directo}} = 32.768 * (T_{\text{fallo}} + T_{\text{lectura}}) + 16384 * T_{\text{escritura}}$$

$$T_{\text{empl_asociativo}} = 32.768 * T_{\text{lectura}} + 512 * T_{\text{fallo}} + 16384 * T_{\text{escritura}}$$

2b. Con asignación en escritura

$$\text{Speed_up} = T_{\text{empl_directo}} / T_{\text{empl_asociativo}}$$

$$T_{\text{empl_directo}} = 32.768 * (T_{\text{fallo}} + T_{\text{lectura}}) + 16384 * T_{\text{escritura}}$$

$$T_{\text{empl_asociativo}} = 32.768 * (T_{\text{fallo}} + T_{\text{lectura}}) + 16384 * (T_{\text{fallo}} + T_{\text{escritura_en_cache}})$$

c) Un programador sugiere la siguiente modificación de código para una cache asociativa por conjuntos de 2 vías con asignación en escritura:

```
#define N 128
struct {
    int A;
    int B;
} AB[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = AB[i][j].A + AB[i][j].B;
```

Asumiendo que el array AB se ubica en la dirección 0x0C000000 y que el array C se ubica a continuación, calcule el número de fallos en lectura y escritura que se producirían, y compárelo con el resultado del apartado b) para la misma configuración de cache.

```
#define N 128
struct {    int A;
           int B;
} AB[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = AB[i][j].A + AB[i][j].B;
```

Las referencias AB[0][0-31] corresponden al primer conjunto de la cache, se colocará en el Bloque 0.

Las referencias AB[0][32-63] corresponden al segundo conjunto de la cache.

Las referencias C[0][0-63] corresponden también al primer conjunto de la cache se colocará en el Bloque 1.

Ahora no existen conflictos continuados entre AB y C.

Fallos de lectura = $128 \times 128 \times 2 / 64 = 512$

Fallos de escritura = $128 \times 128 / 64 = 256$

Número total de fallos = 768

d) ¿Puede obtenerse un resultado similar al del apartado c) con el código original y sin aumentar el tamaño de la cache? Razone la respuesta.

```
#define N 128
int A[N][N];
int B[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = A[i][j] + B[i][j];
```

Solución:

Necesitaríamos que el elemento $C[i][j]$ esté colocado en un conjunto diferente que $A[i][j]$ y que $B[i][j]$.

Para ello deberíamos alargar el array B en 64 elementos. Como es una matriz me resulta más fácil definir un array D[64] entre B y C

```
#define N 128
int A[N][N];
int B[N][N];
int D[64]
int C[N][N];
```